

마감 재고의 가치 있는 순환

Last Dish

CongCong_PodPod

Contact. lastdish.kr

Team. Dongwook Lee(PO) / Yeongjin Kim / Nayoung Kim /
Eunji Yu / Minjung Kim / Ilwon Jung

SAVE MORE



LESS WASTE

Background & Goal.

낭비되는 마감 재고에 가치를 더하고, 소비자와 판매자 모두의 절약을 위한 식품 순환 플랫폼

Healthy & Fit

Environment

환경

- 음식점에서 발생하는 음식물쓰레기는 전체의 40~50% 수준
- 심각한 환경 오염과 온실가스 배출을 유발하는 시급한 자원 낭비 문제

Economy

경제

- 마감 시간마다 판매되지 못한 신선식품이 폐기되는 음식의 비율은 20~30% 수준
- 매장의 고정 손실로 이어지는 소상공인의 고충

Target.

우리 서비스의 예상 사용자

음식 폐기 절감이라는 환경적 가치와 합리적 소비라는 경제적 이점을 동시에 제공하는 핵심 이용자층

User 1



#자영업자

- 버려질 재고를 수익으로 전환
- 폐기로 인한 손실이 적지 않은 자영업자

User 2



#1인 가구

- 양질의 음식을 합리적 가격에 구매를 희망
- 고물가 시대의 대학생 및 1인가구

User 3



#스마트 컨슈머

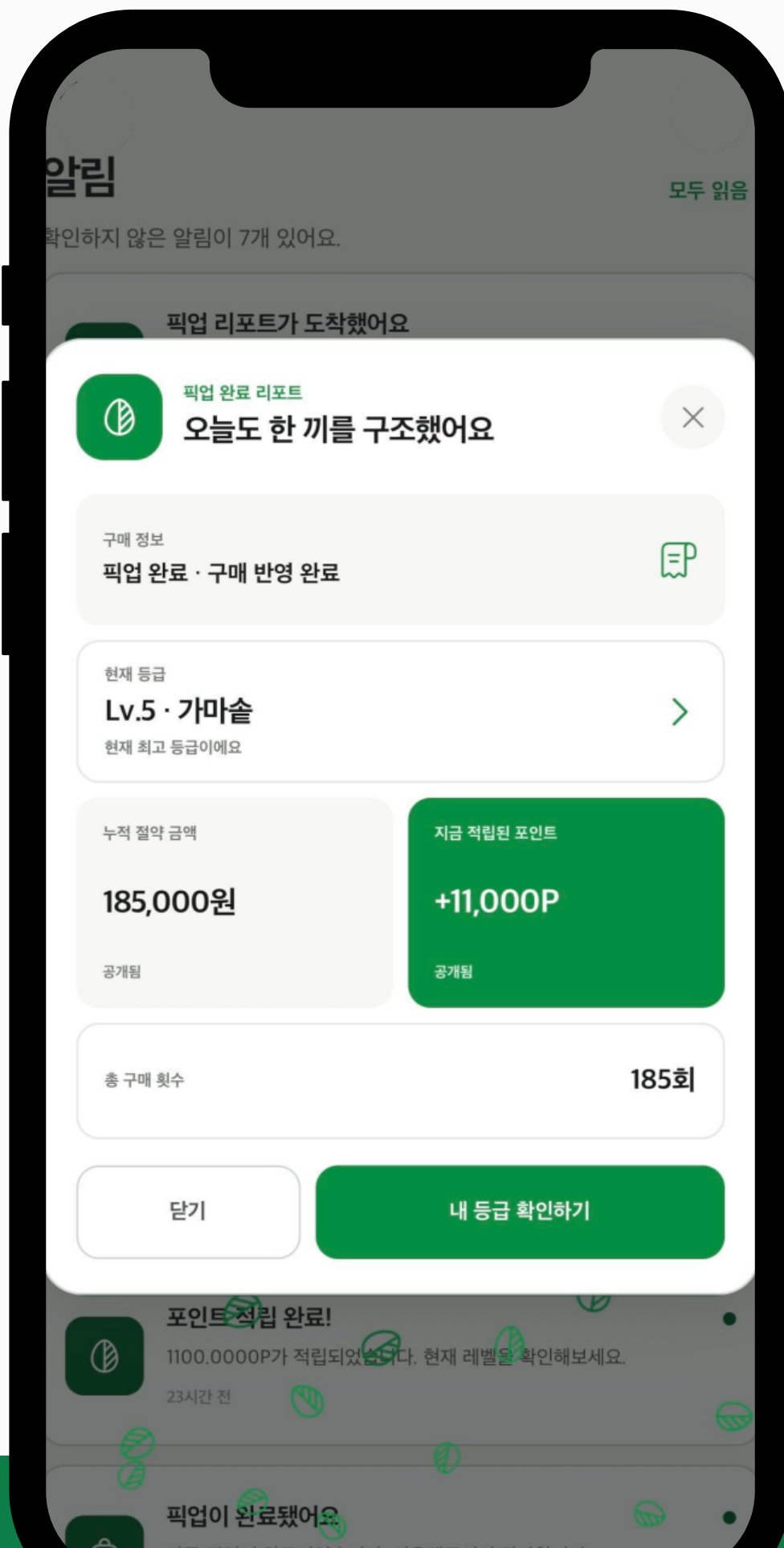
- 환경 보호와 사회적 가치 실현에 동참하려는 소비자 급증

Unique Selling Proposition.

한 번의 픽업이 다음 구매를 만듭니다

마감 상품을 구매하는 순간 **성장·보상·환경 기여**가 함께 쌓입니다.
축적된 혜택은 다시 구매 동기가 되어 지속 가능한 소비 경험으로 이어집니다.

🔄 구매 경험이 다시 방문으로 연결되는 구조



01

STEP 1 · 성장 — 레벨 상승

이용할수록 새로운 혜택

구매 경험이 쌓이면 레벨이 오르고 단계별 맞춤 혜택이 열립니다.

02

STEP 2 · 보상 — 포인트 적립

다음 구매에서 바로 사용

획득한 포인트를 결제에 사용해 보상을 즉시 체감할 수 있습니다.

03

STEP 3 · 환경 — 가치 소비 기록

구매가 만든 가치를 확인

줄인 음식 폐기량을 기록으로 보여주며 지속 가능한 소비를 이어갑니다.

구매자 시나리오

01

사용자 경험을 기능별로
나누어 보여줍니다

CLIP 01 / 07

마감 재고를 찾고 주문부터 픽업까지

사용자가 주변 매장을 발견하고 상품을 결제한 뒤 픽업을 완료하는 핵심 여정입니다.



<https://www.youtube.com/watch?v=awSFNMzN4oA>

Product & Service.

아키텍처 및 구조

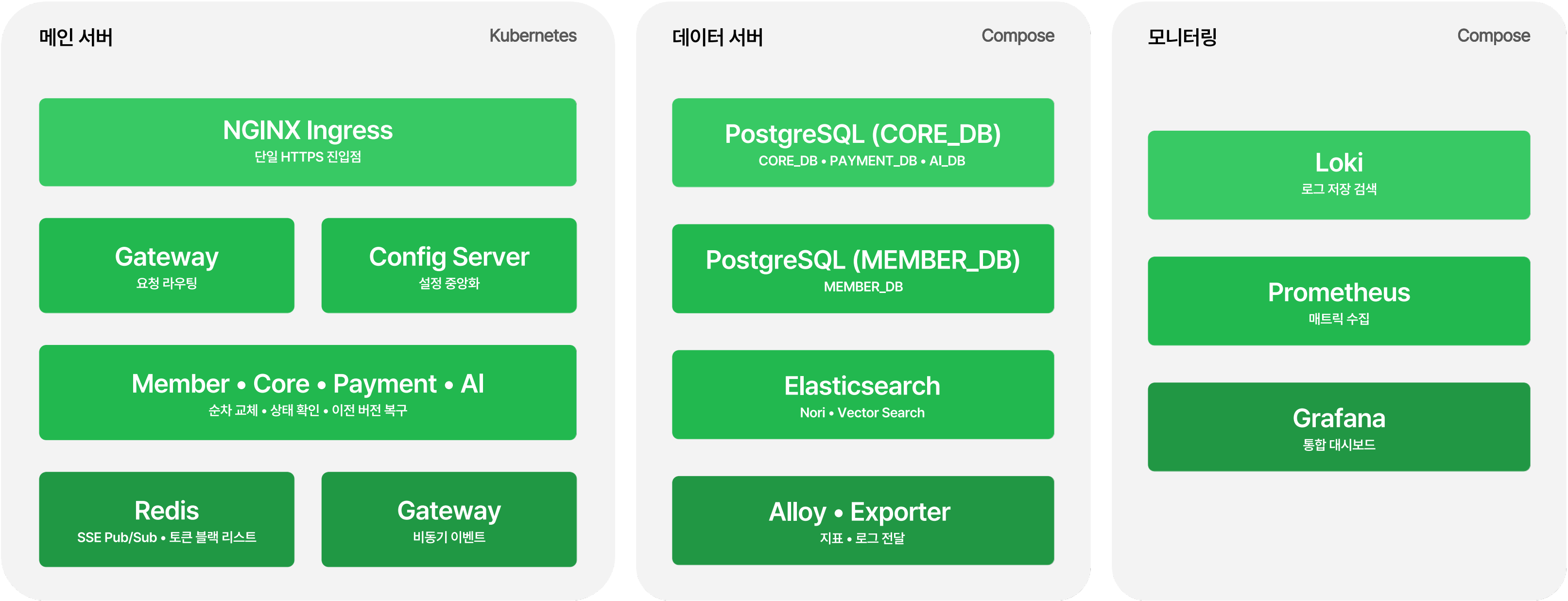
서비스의 인프라 아키텍처와, 각 기술을 어떤 이유로 선택했는지
설명합니다.

01. / 아키텍처 및 구조

역할을 분리해 장애 범위를 줄였습니다.

서비스 실행은 Kubernetes, 데이터는 Private EC2, 관측은 독립 서버에서 담당합니다.

고객 판매자 관리자
Web / Mobile



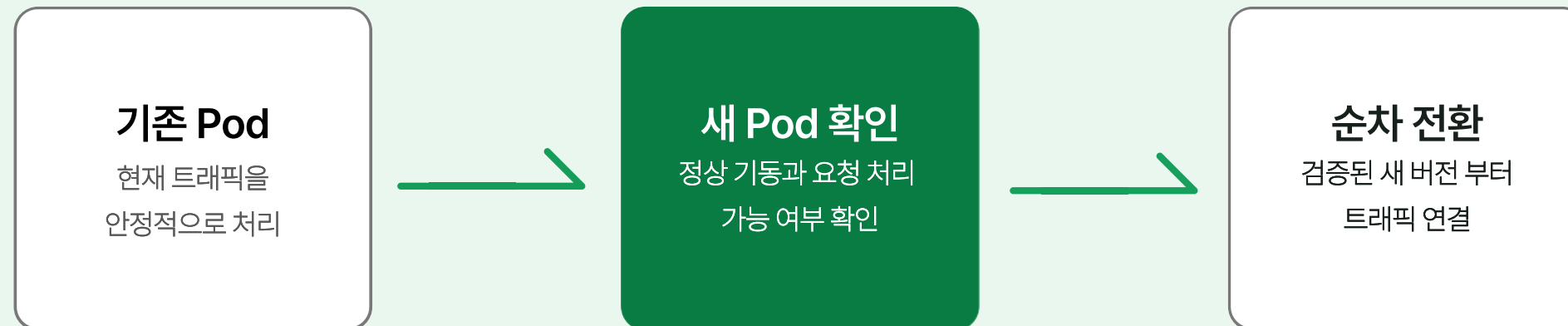
02. / 아키텍처 및 구조

배포 부하를 제어하고 서비스 간 결합도를 낮췄습니다.

서비스는 내부 Pod는 순차 교체하고, 서비스 간에는 순차 배포를 적용해 단일 EC2의 CPU 급증을 제어했습니다.

Kubernetes

새 버전이 검증될 때 까지 기존 버전이 요청을 처리 합니다.



PostgreSQL 17

정합성과 동시 처리

주문 • 결제 • 정산은 데이터 관계와 트랜잭션 정합성이 중요해 관계형 DB가 필요했습니다. PostgreSQL은 프로젝트의 Spring/Jpa•Flyway 구성과 바로 연동할 수 있고, 오픈소스라 별도 라이선스 비용 없이 운영할 수 있어 선택했습니다.

Redis

Pub/Sub TTL

SSE 알림을 모든 Member 인스턴스로 전달하고, 회원 탈퇴 된 Access Token은 만료 시간 동안 블랙리스트로 관리합니다.

Kafka

비동기 이벤트

단건 작업 전달 중심의 큐 보다, 동일한 주문, 매장 이벤트를 정산,포인트,알림,검색 색인이 각자의 속도로 소비하고 장애 후 다시 처리하는 구조가 필요했습니다. 이벤트를 보관하고 소비자 그룹 별 Offset을 관리하는 Kafka가 요구에 적합했습니다.

03. / 아키텍처 및 구조

서비스가 멈춰도 **관측 수단**은 살아있어야 합니다.

관측 서버를 물리적으로 분리하고 로그, 메트릭, 시각화의 책임을 각각 나눴습니다.

**"서비스와 함께 죽는 모니터링은
장애 대응 수단이 될 수 없다."**

메인 서버 내부에 관측 시스템을 두면 서버 장애 시 서비스뿐 아니라 로그 매트릭 조회 수단도 함께 사라집니다.

항상 켜져 있는 별도 물리 서버에 관측 환경을 구성해 장애 중에도 기존 데이터를 조회하도록 했습니다.

메인서버 장애 시에도 기존 로그 매트릭 조회 가능

로그와 매트릭이 하나의 장애 분석 흐름으로 모입니다.

각 도구가 서로 다른 신호를 담당하고 Grafana에서 같은 시간축으로 연결됩니다.

수집 · 대상

메인 서버

Pod · Node · Kubernetes

데이터 서버

PostgreSQL · Elasticsearch

수집 · 저장

Loki

라벨 기반 중앙 로그 검색

Prometheus

Exporter 메트릭 시계열 데이터 수집

연계 · 분석

Grafana

로그와 지표를 동일 시간축에서 비교

장애 분석

원인 서비스 · 시점 · 자원 병목 확인

Loki

무슨 오류가 발생했는가?

Prometheus

시스템 상태가 어떻게 변했는가

Grafana

두 신호가 언제 함께 변했는가

관측 흐름 · Loki가 로그를, Prometheus가 수치를 저장하고, Grafana가 두 정보를 연결해 장애 원인을 보여줍니다.

OBSERVABILITY & k6 LOAD TEST.

기능 구현에서 운영 검증으로

세미 프로젝트에서 구현한 정책과 사용자 경험이
실제 트래픽에서도 유지되는지 최종 프로젝트에서 검증했습니다.

01. / 운영 수용 기준

부하테스트의 기준은 **RPS** 자체가 아니라 **사용자가 주문을 완료**할 수 있는가

운영 상황을 가정해 조회자 · 구매자 · 판매자의 행동 흐름으로 나누고 부하테스트를 설계했습니다.

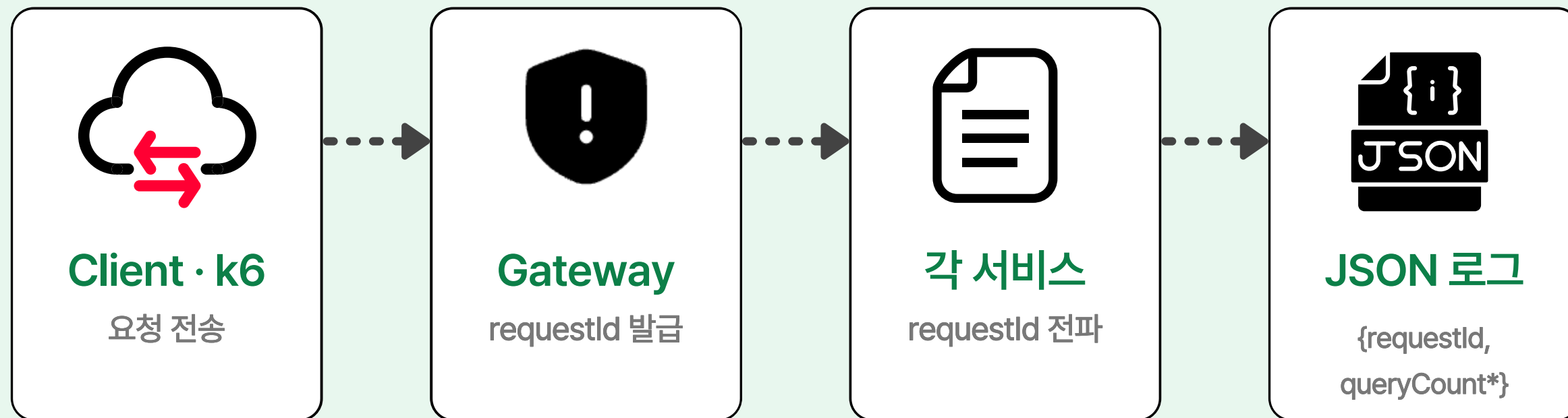


02. / 판단 근거 수집

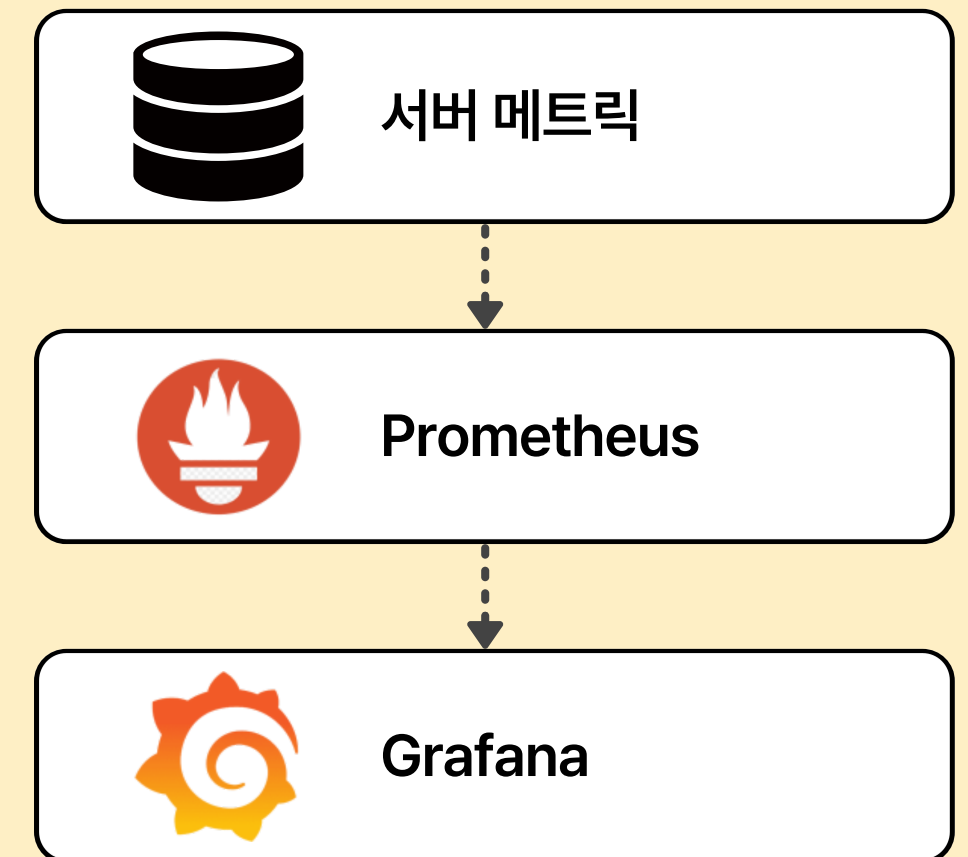
어느 요청이 어디서 어떻게 끝났는지 **추적**할 수 있게 했습니다.

요청 단위 로그와 서버 전체 메트릭으로 서로 다른 범위의 증거를 남겼습니다.

요청 단위 추적



서버 전체 상태 판독



CPU·메모리·DB 연결·요청 처리 상태

03. / 실험 설계

한계·API 비용·재검증을 서로 다른 실험으로 나눴습니다.

하나의 부하 결과로 모든 것을 설명하지 않고, 질문마다 조건과 판정 방법을 분리했습니다.

공동 부하 단위 1 VU = 정해진 사용자 행동 흐름을 반복하는 가상 사용자 1명

01 · 한계

주문의 흐름은 언제 기준을 벗어나는가?

측정 방법 : 주문 요청량을 단계적으로 증가

판정 기준 : 주문 성공률 서버 오류 데이터 정합성

확인결과

180건/분 주문 성공률 100% 모든 기준 통과

240건/분 농친 입력 5건 첫 이상 신호

300건/분 주문 성공률 90.6% 실패 자동 중단

300건/분 실행 중 Gateway OOM 관측

02 · API 비용

포화에 가려진 API별 비용은 무엇인가?

측정 방법 : 1 VU로 API를 개별 호출

연결 기준 : requestId로 요청과 queryCount 연결

확인한 결과

N+1 의심 구간 2곳

쓰기 API의 쿼리 비용 확인

처리 한계가 아닌 API별 비용 계측

03 · 재검증

변경 후 수용 기준을 어디까지 만족하는가?

측정 방법 : 동일한 사용자 흐름으로 재검증

판정 기준 : 01과 같은 수용 기준 적용

확인한 결과

360건/분 주문 성공률 100% 모든 기준 통과

390건/분 주문 성공률 99.72% 모든 기준 통과

420건/분 주문 성공률 97.54% 농친 입력 15건

-5xx 0건, 완주

420건/분에서 재검증 기준 이탈

04. / 측정으로 확인한 개선

한 번의 사용자 흐름을 증폭시키던 요청과 쿼리를 각각 줄였습니다.

응답 성공만으로 보이지 않는 요청과 쿼리 비용을 k6 지표와 운영 로그로 확인했습니다.

클라이언트 요청 증폭

주문 목록 흐름 1회당 호출

k6 흐름 지표

동일한 240건/분 조건

변경 전

23.17회

변경 전 매장별 반복 호출



변경 후

1.26회

변경 후 단일 목록 흐름

불필요한 호출 증폭 94.6% 감소

서버 내부 쿼리 증폭

/orders 서로 다른 매장 50개

1 VU - requestId-queryCount

앞에 부하테스트 2번

변경 전

53쿼리

매장 수만큼 추가



변경 후

3쿼리

매장 수와 무관

데이터가 늘어도 쿼리 수를 상수로 유지

05. / 운영 검증의 결론

서버가 실패하지 않아도 사용자가 주문을 완료하지 못하면 기준 미달입니다.

확인된 결과와 아직 측정하지 않은 효과를 구분해 결론을 내렸습니다.

1라운드

180건/분

마지막 기준 통과



개선 후 재검증

390건/분

동일 조건 3회 모두 통과



첫 기준 이탈

420건/분

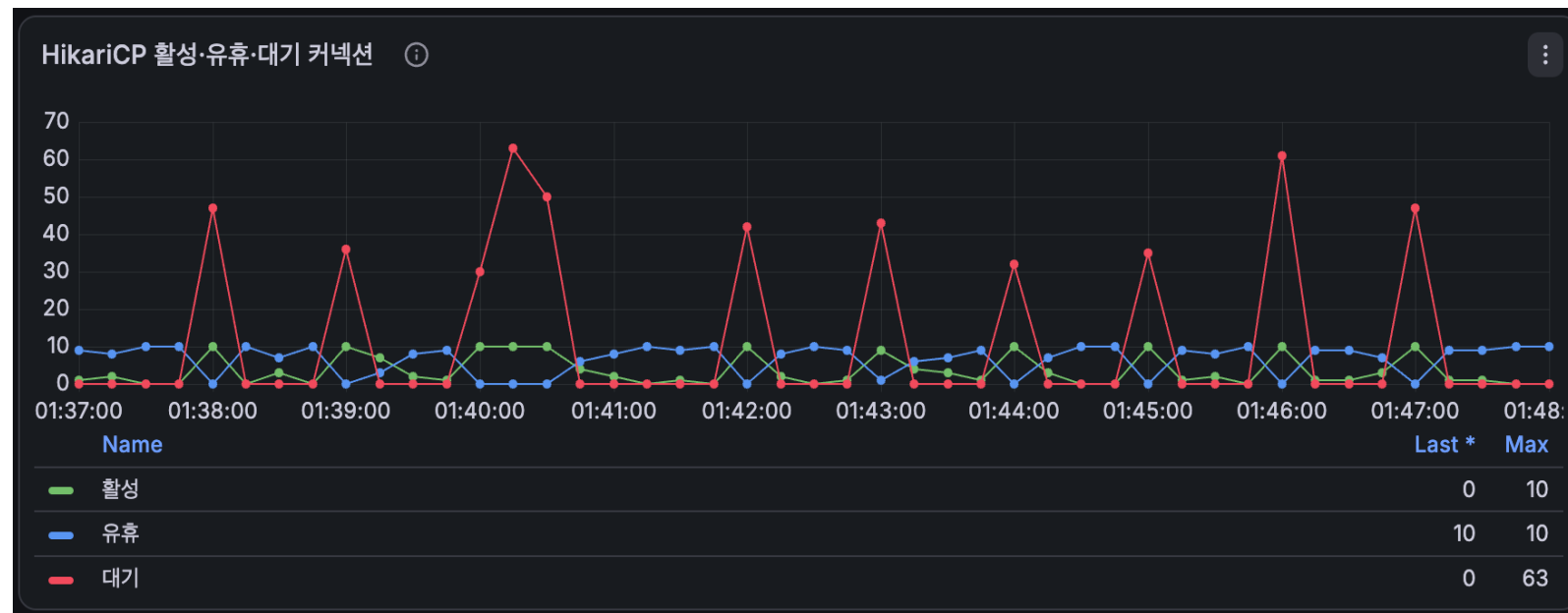
주문 성공률 97.54%



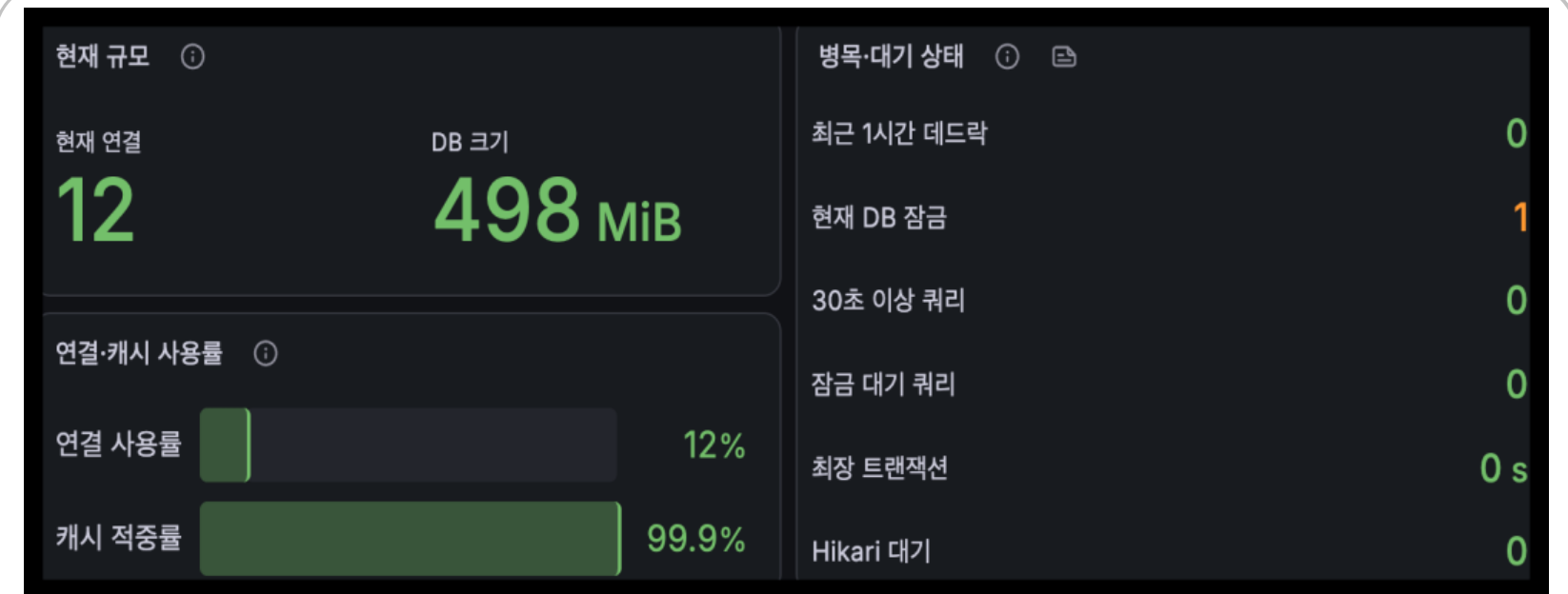
커넥션 풀 개선 및 VU 구조 수정

510건/분

부하 도구의 VU가 먼저 부족



Grafana : 활성 최대 10 대기 최대 63



DB 포화 신호 없음 : 연결 12% 캐시 99.9% 데드락 잠금 대기 0

DB를 우선 병목 후보에서 제외하고, 커넥션 풀에서 요청 대기가 발생까지 검증 범위를 좁혔습니다.

Product & Service.

서비스 간 통신 방식

이벤트의 발행과 처리뿐만 아니라, 완료된 데이터의 폐기까지
하나의 운영 생명주기로 관리하게 됐습니다.

01. / 서비스 간 통신 방식

통신 기술이 아니라 **응답 시점과 중요도**를 기준으로 선택했습니다

모든 호출을 하나의 방식으로 통일하지 않고, 현재 응답, 후속 처리, 핵심 데이터 보호라는 세 질문으로 구분했습니다.

01 · HTTP

지금 결과가 중요한가?

현재 요청의 성공 여부나 응답 데이터에 호출 결과가 직접 필요합니다.

현재 응답과 함께 완료

CLIENT



HTTP
SERVICE

02 · KAFKA EVENT

후속 작업을 분리할 수 있는가?

반드시 처리해야 하지만 현재 사용자 응답과 묶을 필요는 없습니다.

응답 이후 소비자별 처리

Producer



Kafka
Event

03 · EVENT + HTTP

핵심 데이터라 한 경로로 부족한가?

상점 검색 데이터는 서비스 핵심 로직이므로 두 경로가 서로 장애를 보완합니다.

AI 상점 검색 데이터 갱신

변경
Event

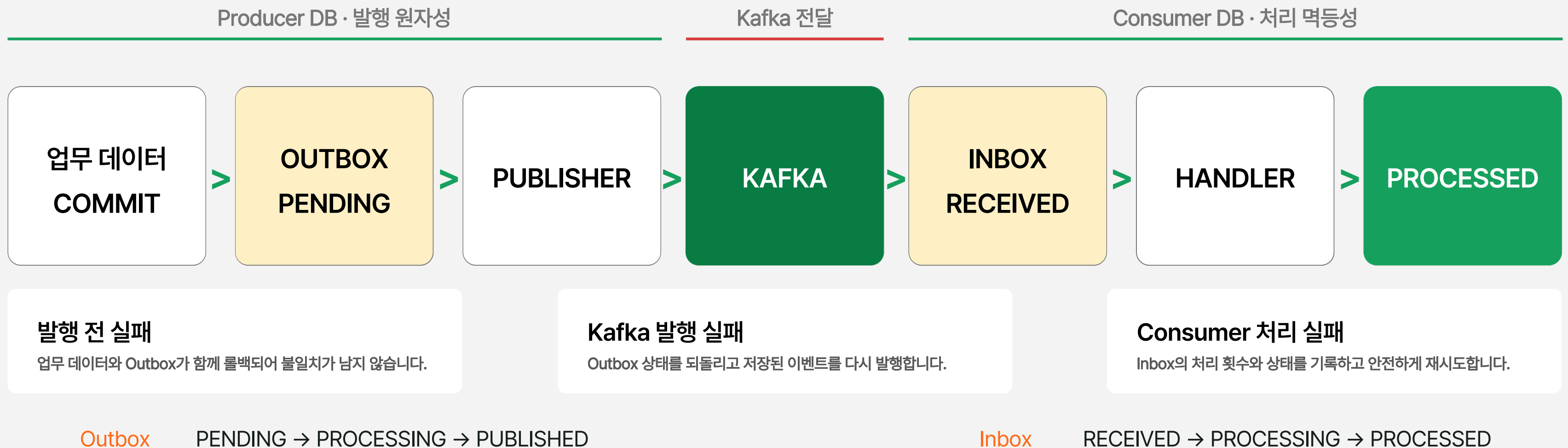


원본
HTTP 조회

02. / 네 가지 중 대표 경로

신뢰성 요구가 가장 높은 **Outbox + Inbox**를 대표로 설명합니다

주문, 결제, 정산 처럼 유실과 중복 반영에 민감한 이벤트는 발행부터 처리 완료까지 각 경계의 상태를 기록합니다.



03. / 발행 신뢰성

Outbox로 업무 저장과 이벤트 기록을 하나의 트랜잭션으로 묶었습니다.

데이터는 저장됐지만 Kafka 발행이 실패해 후속 서비스가 변경 사실을 모르는 상황을 방지합니다.

DB와 Kafka를 따로 쓰면

두 작업 사이에 장애가 발생할 수 있어 하나의 성공만으로 전체 성공을 보장할 수 없습니다.

업무 DB 커밋

≠

카프카 발행 실패

주문은 완료됐지만 결제·정산 이벤트는 전달되지 않음

업무 데이터와 Outbox를 함께 저장

주문 결제 데이터 변경

Outbox 이벤트 PENDING

동일한 데이터베이스 트랜잭션 : 같이 COMMIT 또는 같이 ROLLBACK

1

Outbox 선점

>

2

Kafka 발행

>

3

성공 확인

>

4

PUBLISHED

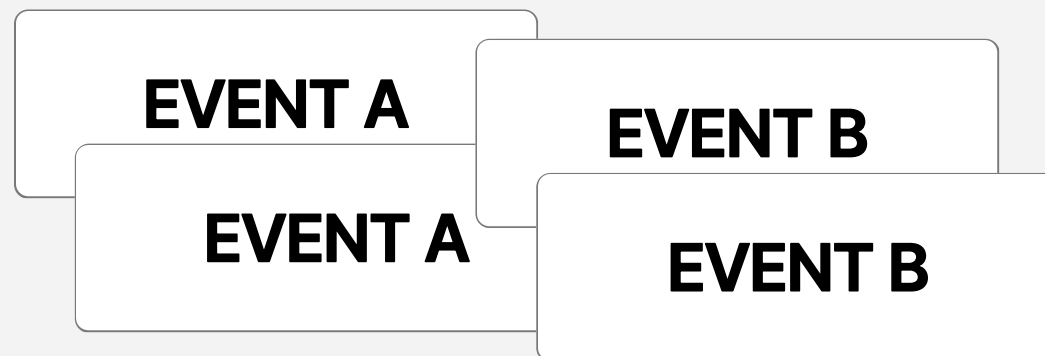
PENDING → PROCESSING → PUBLISHED 실패 시 PENDING 복귀 또는 재시도 소진 후 FAILED

04. / 소비 신뢰성

Inbox로 중복 전달과 처리 실패를 비즈니스 중복 반영으로 이어지지 않습니다

Kafka의 재전달을 허용하면서 이벤트 처리 상태를 소비자 데이터베이스에 기록합니다.

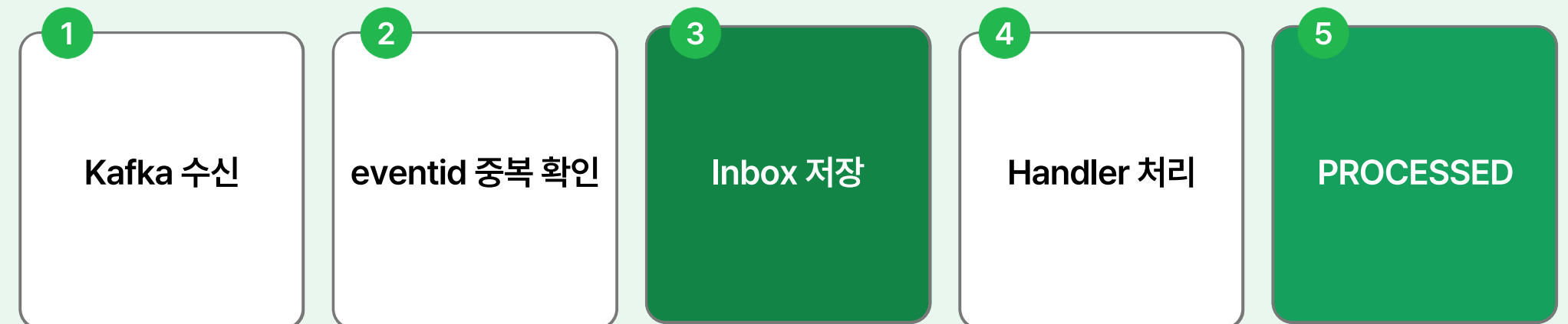
이벤트는 다시 올 수 있습니다.



중복 결제, 중복 포인트

Consumer 처리 후 ACK 전에 장애가 발생하면 같은 메시지가 다시 전달될 수 있습니다.

먼저 저장하고, 처리 상태를 추적합니다.



멱등성

같은 consumerId와 eventId는 한 번만 저장

실패 재시도

처리 상태와 횟수를 기록해 재처리

순서 보호

Aggregate 버전으로 오래된 상태를 SKIPPED

EventHandlerRegistry · (consumerId, eventType)으로 처리 목적과 이벤트 종류를 분리해 동일 이벤트도 여러 목적에서 독립 처리합니다.

05. / 이벤트 처리 정책

중복은 차단하고, 상태 변경은 **최신 버전**만 반영했습니다

이벤트 ID 기반 멍등성을 공통 적용하고, 데이터 성격에 따라 모든 이벤트 반영과 최신 상태 우선 정책을 구분합니다.

멍등 처리

IDEMPOTENT

금액 증감이나 포인트처럼 서로 다른 이벤트를 모두 반영해야 하는 처리입니다.



EventHandlerRegistry · (consumerId, eventId)를 Inbox 식별자로 사용합니다. 사전 조회와 DB Unique 제약으로 동시 중복 저장도 차단합니다.

최신 상태 우선

IDEMPOTENT_LATEST_WINS

회원·상품 상태처럼 과거 변경보다 가장 최신 Snapshot이 중요한 처리입니다.



버전 진행 행을 잠근 뒤 **incoming ≤ lastProcessedVersion**이면 Handler를 호출하지 않고 `OLDER\_THAN\_LAST\_APPLIED`로 기록합니다.

처리 보장 · 재전달은 허용하되 같은 이벤트의 중복 반영을 막고, 상태형 이벤트는 늦게 도착한 과거 버전이 최신 데이터를 되돌리지 못하게 했습니다.

06. / 선택 가능한 전달 경로

대표 경로에서 저장 단계를 선택적으로 줄여 4가지 경로로 확장했습니다

Outbox는 발행 보장, Inbox는 소비 재시도를 담당하며 이벤트 중요도와 지연 요구에 필요한 단계만 선택합니다.

Outbox + Inbox

신뢰성 최우선

발행 유실과 소비 실패를 모두 보호

주문 · 결제 · 예치금 · 재고 · 정산

Direct + Inbox

소비 처리 보호

발행 유실은 허용하고 중복·순서·재시도 관리

복구 가능한 보조 데이터 동기화 · 통계

Outbox + Direct

발행 보장 + 즉시 처리

이벤트는 놓치지 않고 소비 저장 단계는 생략

인앱 알림 · SSE

Direct + Direct

속도 최우선

일부 유실을 허용하고 DB 작업 최소화

로그 · 분석 · 일부 통계

신뢰성은 높지만
DB작업도 늘어 납니다.

Outbox + Inbox

혼합 경로

Direct + Direct

Outbox-Inbox는 INSERT, 상태 UPDATE, 선점, 재시도와 정리 작업을 추가합니다. 트래픽 증가 시 병목이 될 가능성을 줄이기 위해 중요한 이벤트에만 신뢰성 비용을 사용합니다.

공통 EventMessage

Listener → EventHandlerRegistry(consumerId, eventType)

전달 경로와 비즈니스 Handler 분리

07. / 운영 트러블 슈팅

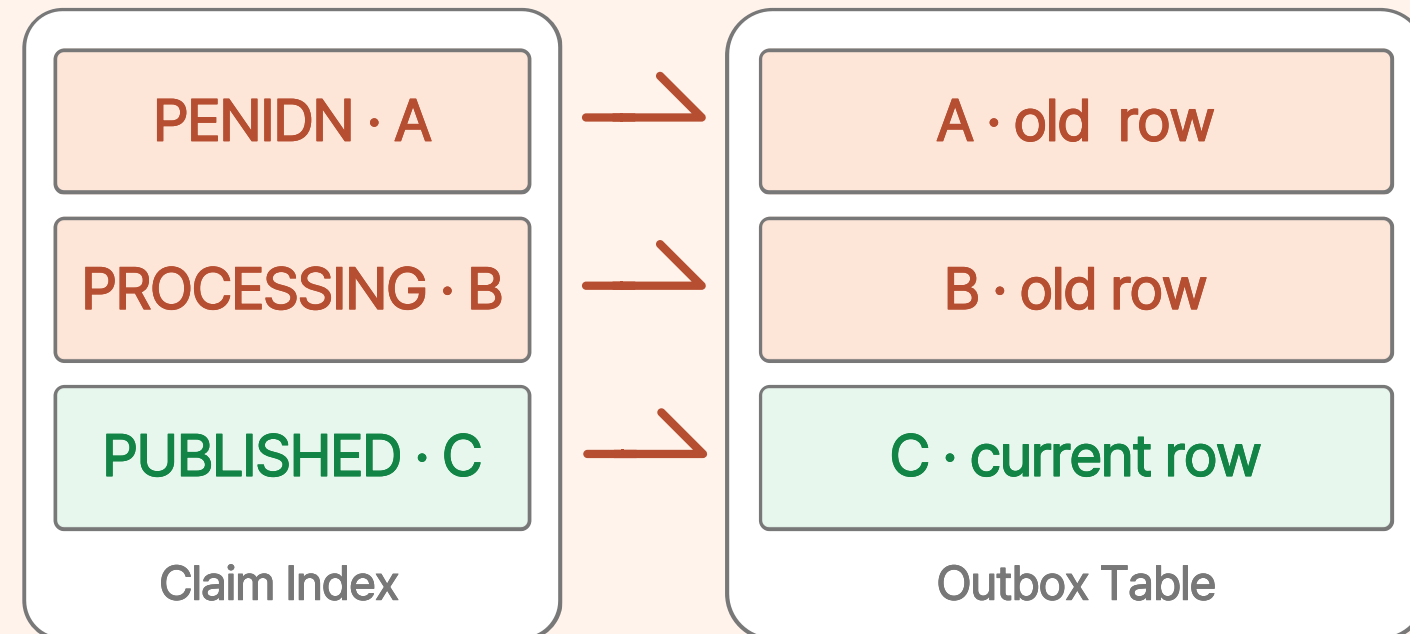
처리할 이벤트는 0건인데, Claim은 죽은 인덱스 항목을 읽고 있었습니다

완료 행 누적과 MVCC 흔적을 분리해 해결하고, 운영 60,000건으로 조회·정리·Vacuum의 전체 생명주기를 검증했습니다.

01. 기존 읽기 경로

인덱스가 죽은 행 버전까지 가리키고 있었습니다

상태 UPDATE는 이전 행을 즉시 덮어쓰지 않습니다. Vacuum 전까지 오래된 인덱스 주소가 남아, 빈 Claim도 테이블 가시성을 확인한 뒤 버렸습니다.



Claim 결과 0건 · 확인 비용 820 buffers / 2.688ms

02. 역할별 해결

한 가지 튜닝이 아니라 세 책임으로 나뉘었습니다

1

부분 Claim 인덱스

PENDING-PROCESSING만 포함해 살아 있는 PUBLISHED를 조회 경로에서 제외

2

Autovacuum 기준 완화

Outbox 20% → 2%로 낮춰 상태 변경의 dead entry를 더 일찍 회수

3

완료 데이터 Cleanup

Outbox는 60초마다 1,000건, Inbox는 3일 보관 후 점진 삭제

03. 운영 검증

같은 60,000건에서
읽기 범위가 사라졌습니다

Claim 인덱스 3,312kB → 8kB

빈 조회 Buffer 820 → 4

빈 조회 시간 2.688ms → 0.026ms

Cleanup 실측 1000건 / 60초

임계치 초과 후 Autovacuum 자동 실행
두 번째 Cleanup에서 임계치를 넘자 죽은 행을 정리를 확인했고, 다음 Cleanup부터 다시 누적됐습니다.

Product & Service.

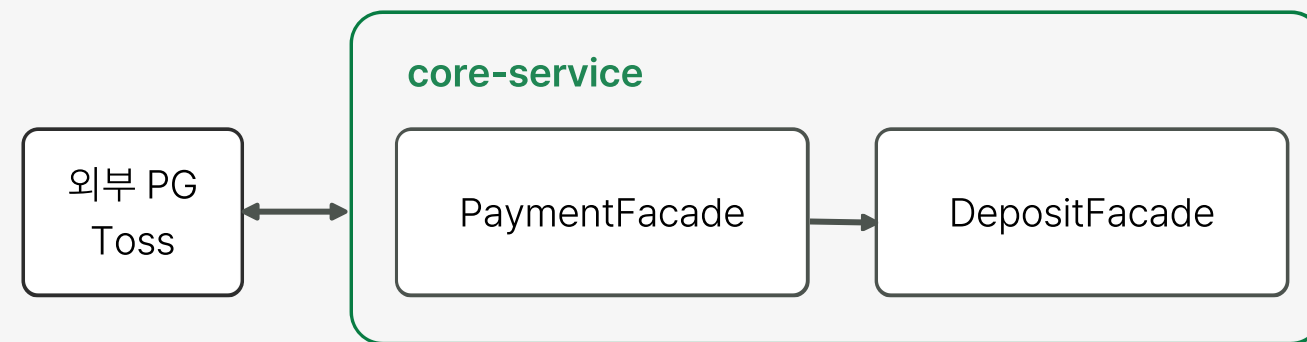
Payment Deposit Point Level

결제 도메인을 물리적으로 분리하고,
방치된 상태 값과 중복 이벤트 문제를 처리했습니다.

01. / payment-service 분리

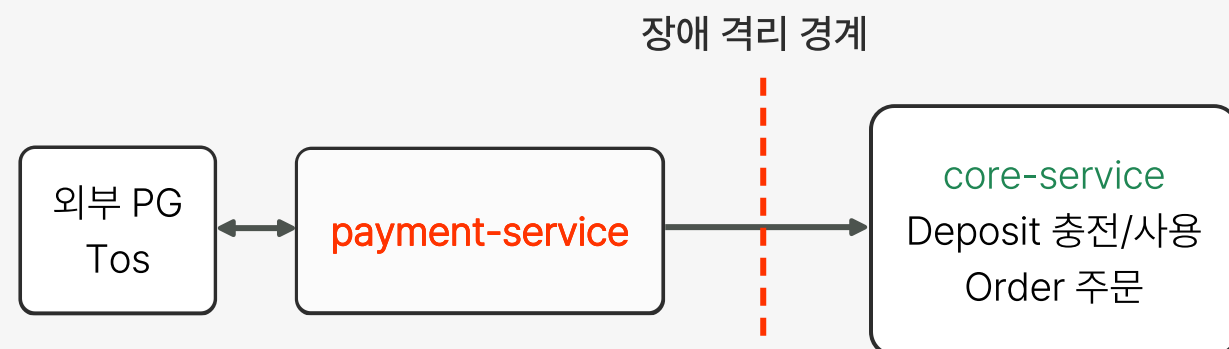
주문·예치금 코어까지 전파되던 외부 PG 장애극복

core-service 내부 + 동기식 호출



PG 장애·지연 전파 → core-service 스레드·커넥션 고갈

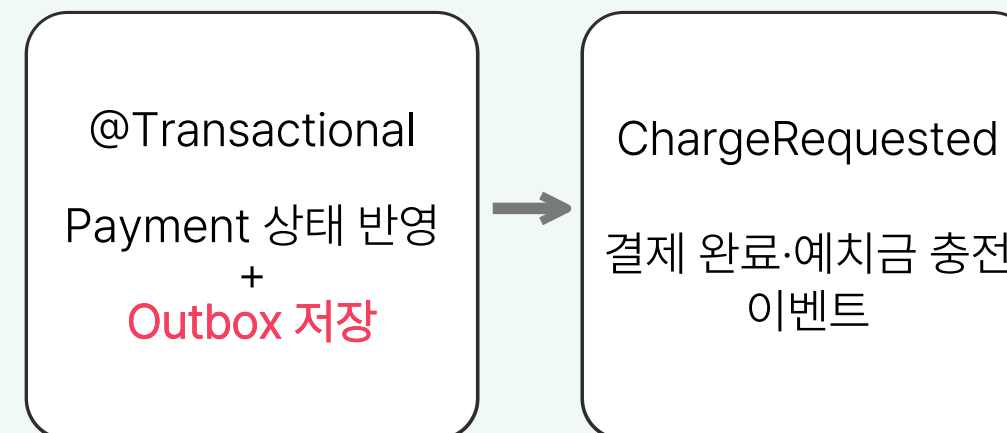
payment-service 분리 + 비동기



PG 장애 발생해도 payment-service → 주문·예치금 사용은 영향 없음

Kafka 이벤트 비동기 호출
Outbox + Inbox 패턴 적용

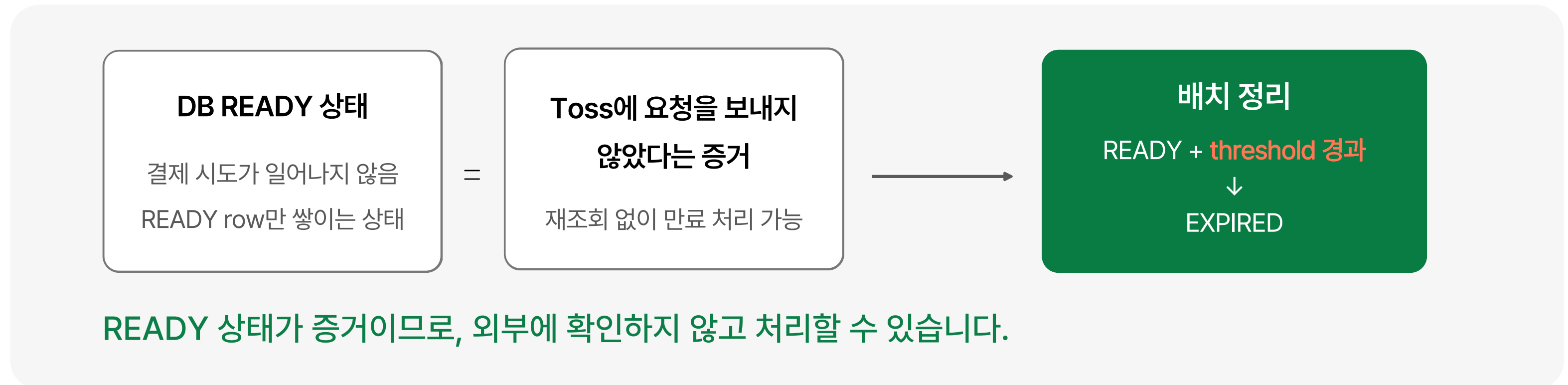
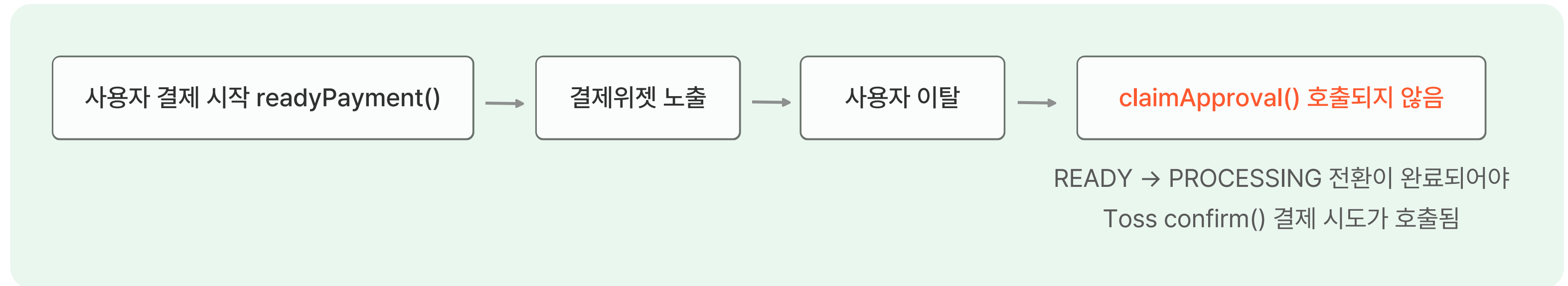
payment-service



02. / 방치된 결제 상태 처리

방치된 결제 상태 처리 — **READY** 상태

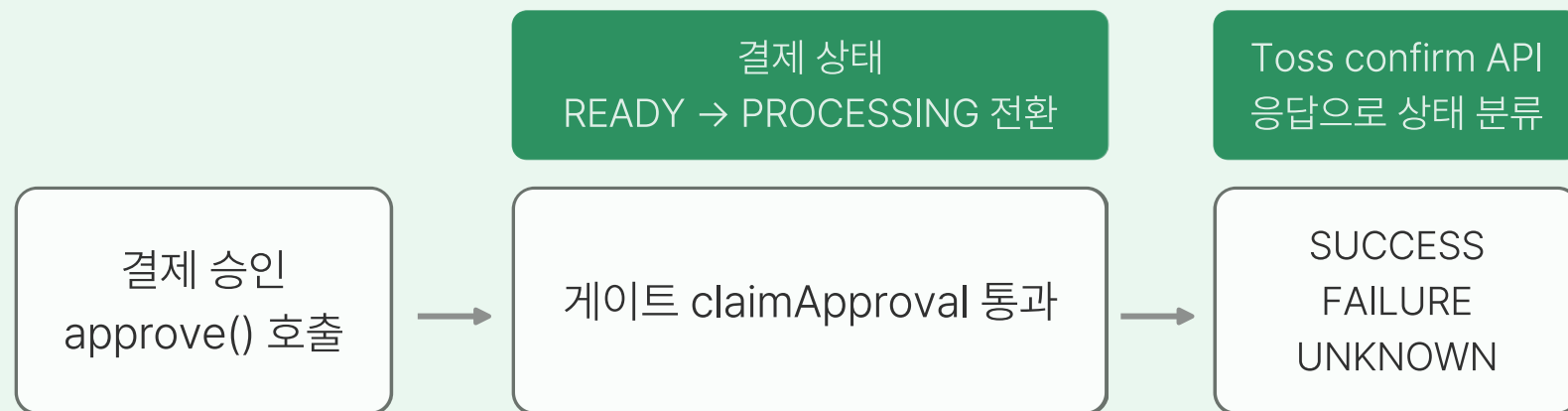
방치된 READY 상태의 결제 건은 Toss에 묻지 않고 배치로 자동 만료 EXPIRED 처리합니다.



03. / 방치된 결제 상태 처리

방치된 결제 상태 처리 — PROCESSING 상태

방치된 PROCESSING 상태의 결제 건은 Toss 재조회 API로 승인 결과를 확인하여 최종 기록합니다.

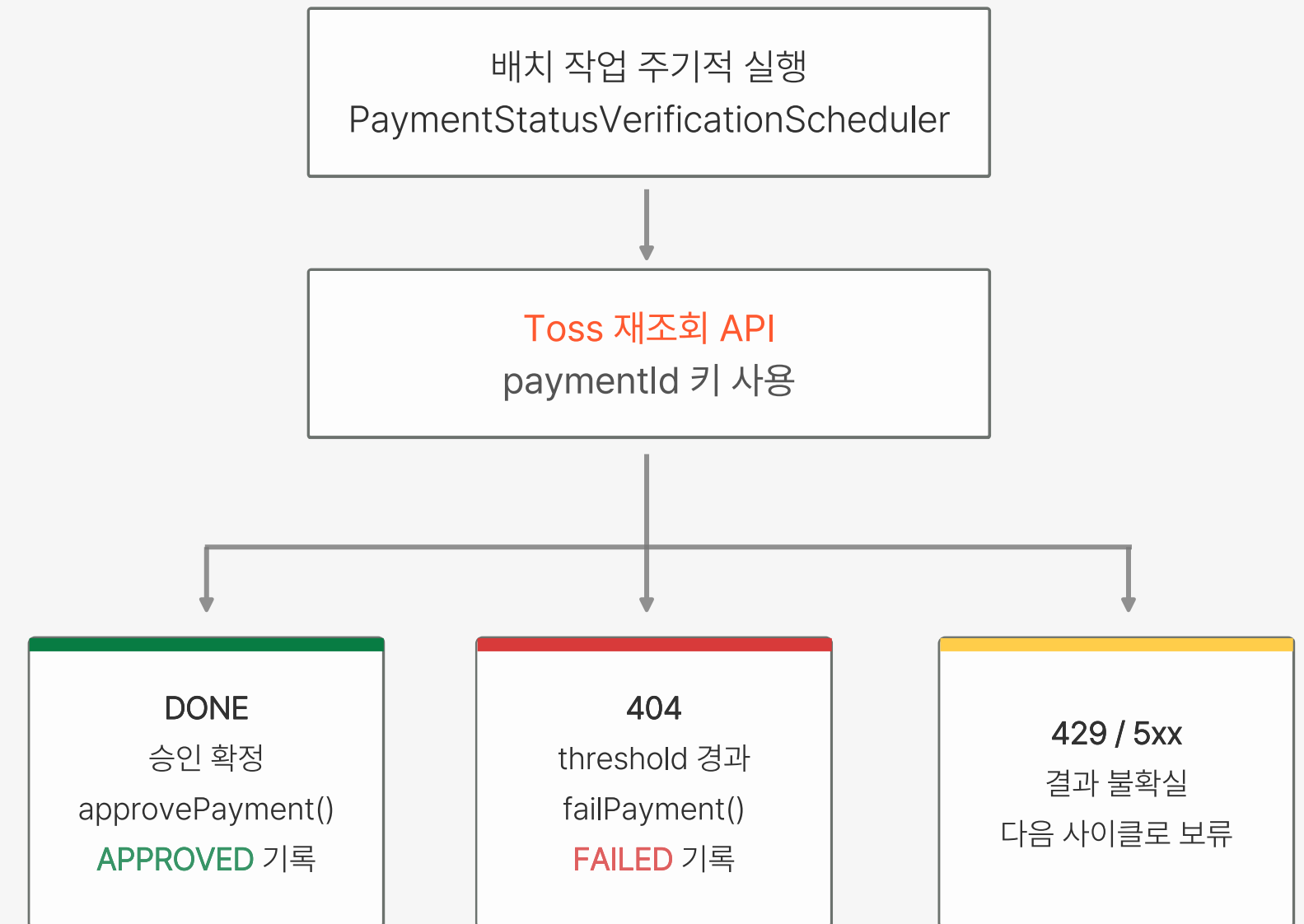


결제 상태가 **PROCESSING**에서 갇히는 2가지 CASE

case 1. UNKNOWN 무응답·timeout — Toss 응답 지연·실패로 응답을 받지 못함

case 2. SUCCESS PG 결제 승인은 완료 — DB 상태 반영 중 예외 발생

통신이 시작된 이상, 불확실한 결과에 대해 임의 취소·재시도 할 수 없습니다.



요청이 진행된 통신은 다시 물어봐서 확정지어야 합니다.

04. / Point & Level 기능 추가

Point & Level — 적립과 등급 계산

Inbox 이벤트 중복 처리 문제를 트랜잭션 경계와 멍등성 게이트로 막았습니다.

problem

Order에서 네트워크 재시도 등으로 OrderPickedUp 이벤트를 같은 주문에 대해 다른 eventId로 중복 발행하는 문제

reason

- Inbox는 eventId 기준으로 중복을 판단함
- 새 이벤트로 통과 → 포인트 적립·등급 계산·알림 이벤트 발행 중복 실행

solution

- hasAlreadyEarned() 메서드 : orderId로 처리 여부 확인
- 트랜잭션 맨 앞에 추가 → 이미 처리된 주문이면 적립·레벨·알림 실행 X

ORDER

픽업 완료 시 OrderPickedUp 이벤트 발행 (Outbox)

OrderPickedUp 이벤트 구독 (Inbox)
실패 시 Inbox 최대 5회 재시도

PointFacade — @Transactional

Point → Level 락 순서 고정

hasAlreadyEarned() 게이트
통과 시 포인트 적립 → 등급 계산 및 반영

MemberReward 이벤트 발행 (Outbox)
사용자 리워드 처리 완료 알림

NOTIFICATION

이벤트 구독 (Direct) + 사용자 알림 발송

Product & Service.

Dish

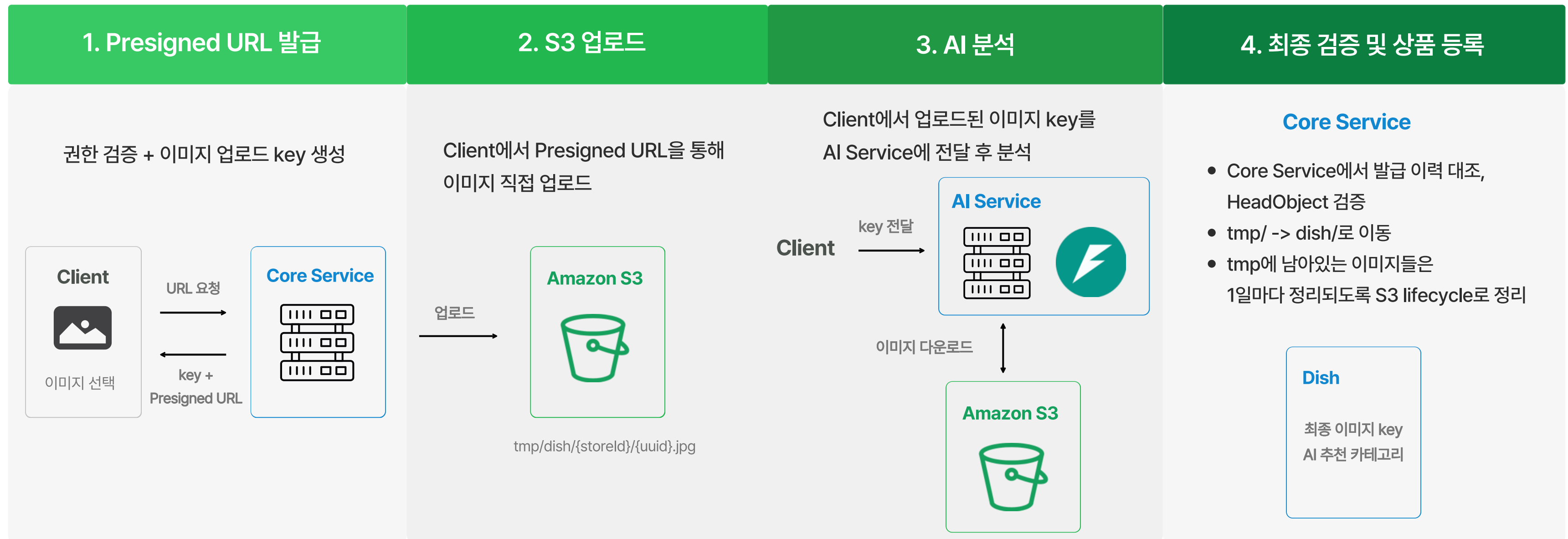
Cart

Order

제품의 비즈니스 정책과 도메인 간 연관성을 고려해
주문 프로세스를 구현했습니다.

01. / S3 이미지 업로드

Presigned URL 기반 상품 이미지 업로드 및 AI 분석 구조



성과

직접 업로드

Core Service를 거치지 않고 S3에 직접 업로드하여 서버 부하 최소화

권한 통제

Presigned URL과 IAM Role 기반 권한 통제를 적용하여 안전한 이미지 업로드 및 AI 분석 구조를 구축

검증 후 저장

발급 이력과 S3 객체 검증 후 임시 디렉토리에서 실제 서비스 경로로 이동하여 안전하게 저장

02. / 회원 정보 조회

주문 생성 시 회원 정보 조회 전략 개선

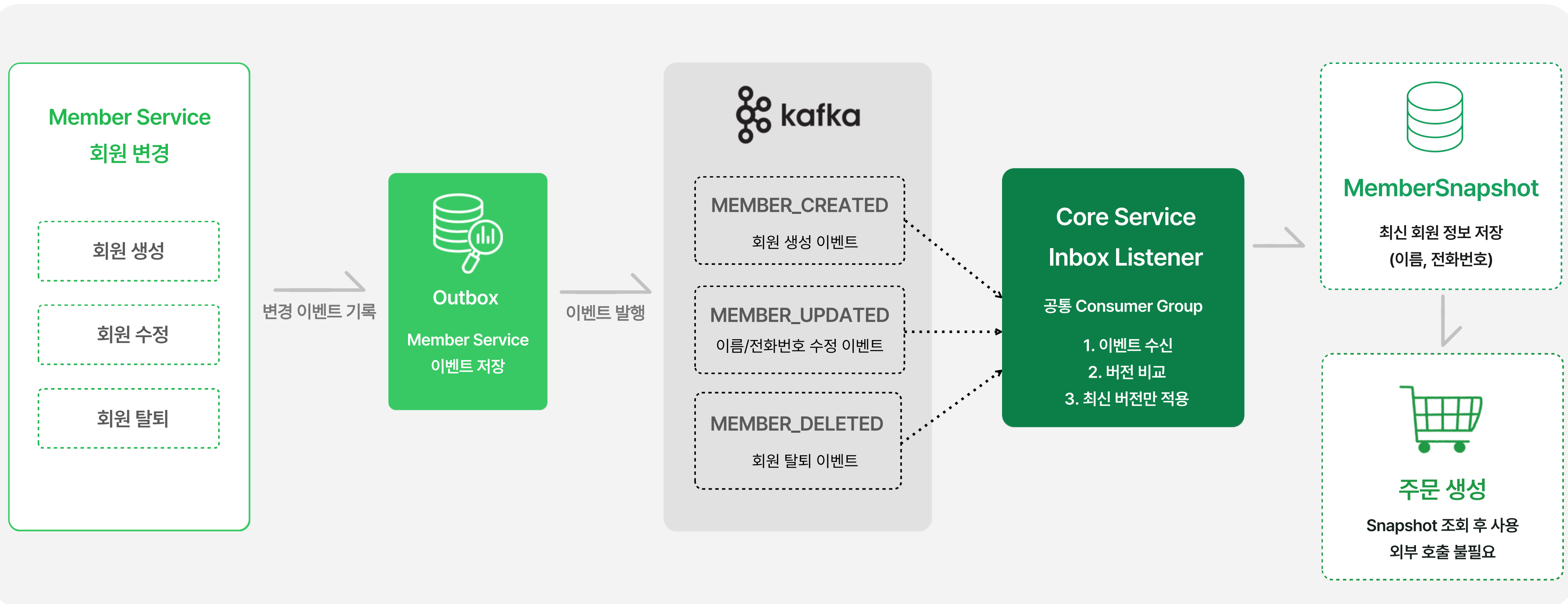
MSA 구조에서 주문 생성 후 픽업 시 정보 조회를 위한 회원 이름과 전화번호 저장



03. / 회원 정보 동기화

Event 기반 회원 정보 동기화

Member Service는 변경 이벤트를 발행하고, Core Service는 하나의 리스너로 모든 이벤트를 처리하여 Snapshot 동기화



04. / 주문 및 픽업 기능 개선

주문 비즈니스 로직 개선



주문 픽업 완료, 노쇼 이벤트

EVENT

ORDER_NO_SHOW

ORDER_PICKED_UP



SETTLEMENT

LEVEL

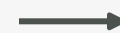
DOMAIN



주문 노쇼 처리 스케줄러

SCHEDULE

주문 픽업 마감시간
스케줄링



EVENT

픽업 마감 후 미수령 주문
자동 NO_SHOW



픽업 알림

판매자

주문 완료
주문 취소

주문자

주문 접수
주문 반려

픽업 시작 시간
픽업 마감 15분 전
판매자 픽업/노쇼 처리



주문 직전 상품, 장바구니 검증

problem

1. 상품 정보 변경 시 장바구니 상품 정보가 자동 동기화되어 실제 결제 가격이 다르게 결제됨
2. 나의 주문 내역 조회 시 삭제된 상품 조회 불가

solution

DISH

UPDATE 시 기존 상품 SOFT DELETE + 새로운 상품 CREATE

DISH

SOFT DELETE + ON_SALE 검증

CART

주문 시 장바구니의 상품 정보 조회
최신 상품이 아니면 EXCEPTION 발생



ORDER

최신 상품으로 주문 생성

판매중인 상품이
아닙니다.

Product & Service.

Settlement

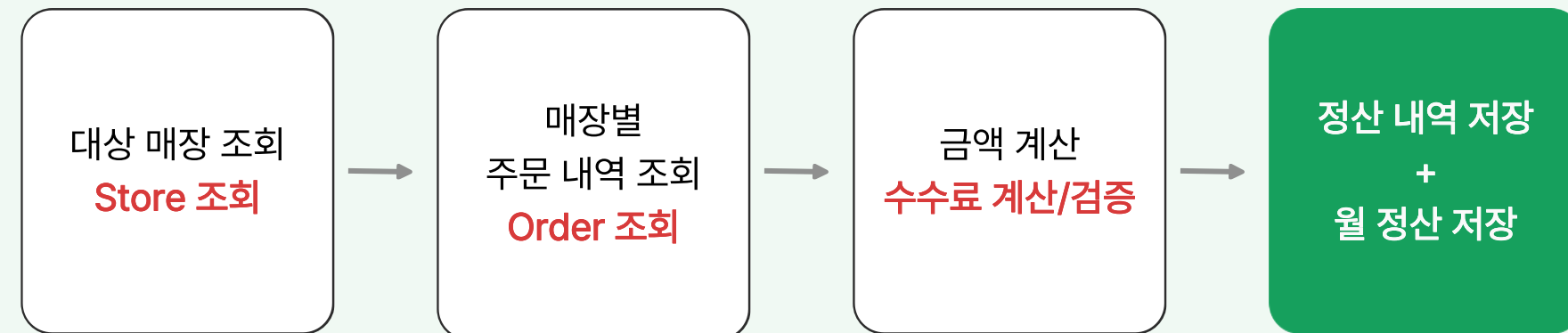
비즈니스 정책과 서비스 자원을 고려한 설계로
정산 기능의 안정성과 성능을 끌어올렸습니다.

01. / 개선 전 정산 흐름 및 구현

초기 정산 방식

Tasklet 기반의 단일 반복문, 정산 시점에 매장별로 주문 내역을 조회해 정산을 진행

월 정산 흐름



Tasklet.execute() 내부 흐름

전체 대상 조회 → for문 순회 → 매장별 서비스 호출 → 결과 카운트

적용된 정산 규칙

- 월 1회 지정일에 직전 달의 완료된 주문에 대해 정산
- 매장별로 주문 완료 내역을 불러와 한 번에 정산
- 각 매장을 별도 트랜잭션으로 분리
- 정산 테이블 매장과 정산 월의 조합으로 유일성 제약
- 정산 내역 테이블은 order_id 유일성 제약

방식 선택 이유

- 매장 조회, 주문 조회, 정산, 저장의 단순한 흐름
- 이미 주문이 완료되어 변경될 일이 없는 데이터
- 전체 정산 처리 흐름을 서비스에 맞게 구현 가능

02. / 문제 상황 및 해결

기능 테스트 중 2가지 문제점 발생

메모리 부족 문제와 처리 속도 지연 및 저장 성능에 대한 병목 발생

엔티티 누적으로 인한 OutOfMemory 발생

100개 매장 x 1000건 = 10만 건 정산 진행 중 52개 매장 처리 후 OutOfMemory 발생

원인

주문 내역을 엔티티로 조회하면서 Tasklet 트랜잭션의 영속성 컨텍스트에 누적, GC 발생 시 객체가 회수되지 않아 정산이 종료될 때까지 메모리를 차지

해결

Projection 조회를 통해 필요 컬럼만 반환하여,
영속성 컨텍스트 관리에서 제외시켜 GC 발생 시 메모리 Heap 복원

상세 내역 반복 Insert로 인한 속도 지연

300개 매장 x 1500건 = 45만 건 정산이 약 8분 소요

원인

saveAll()이 정산 내역별로 INSERT를 반복해 네트워크 왕복이 주문 건 수만큼 증가

해결

여러 상세 내역을 단일 SQL로 쿼리 수를 축소해
쓰기 병목을 제거하고 처리 시간을 300개 매장 8분에서 1분으로 단축

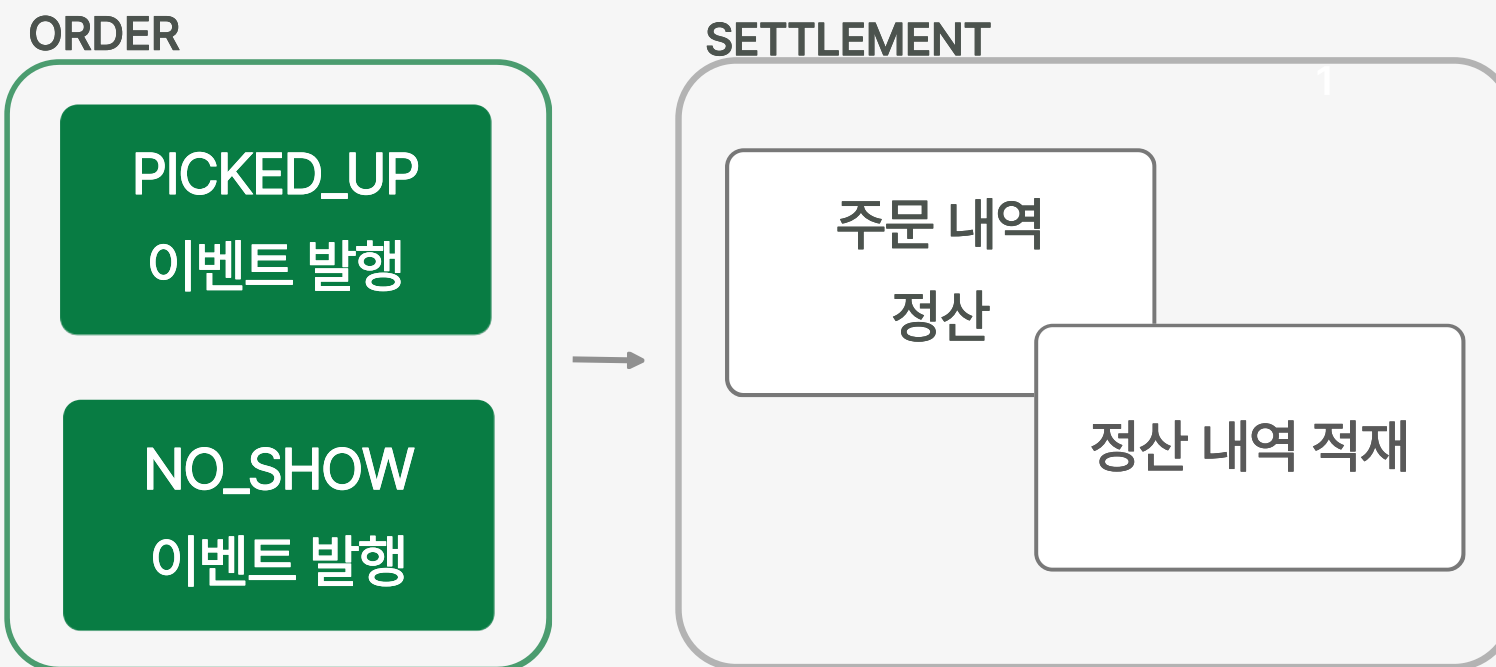
03. / 정산 방식 전환

문제는 해결되었지만, 여전히 남은 개선 지점들

정산 기능이 서비스의 CPU 가용치의 60%를 차지하고, Tasklet 전체 트랜잭션으로 인해 쓰이지 않는 DB 커넥션이 존재

주문 완료 시 정산 내역을 미리 적재

주문 완료 시 발행된 이벤트를 구독해 정산 내역을 미리 적재해두도록 변경,
정산 시점에는 월 정산 내역의 총 집계만 진행

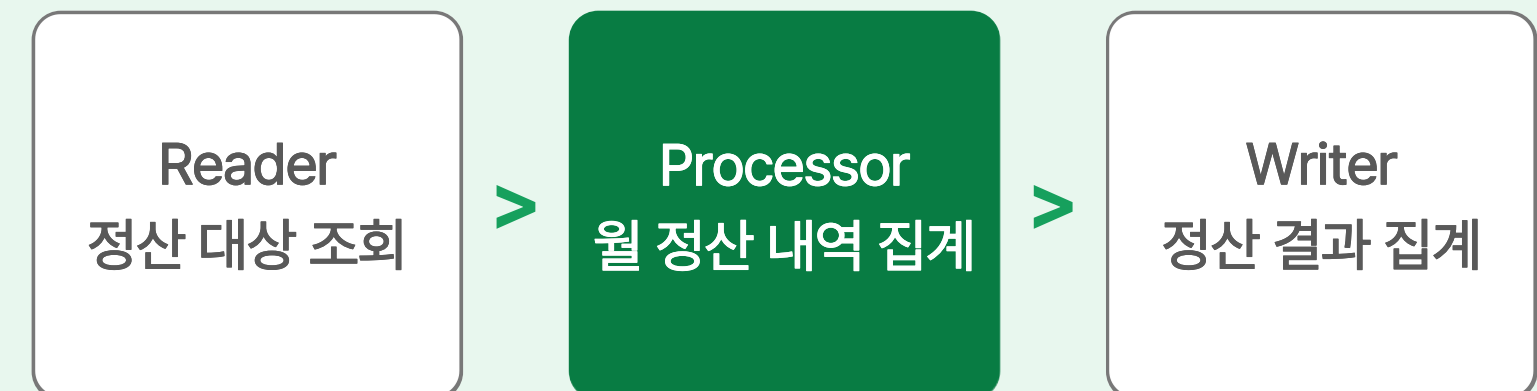


Outbox, Inbox 패턴으로 누락을 방지하고, 정산 내역의 order_id 유니크 제약으로 중복을 방지

정산 기능의 CPU 사용량 27.7%로 감소

매장별 Chunk 방식으로 전환

기존 Tasklet execute() 내부의 단일 반복문 방식에서 매장별 Chunk 방식으로 전환해 트랜잭션을 분리



Reader에서 한 번에 100개씩 월 정산 대상을 조회,
읽어온 정산 대상을 매장 단위로 Chunk 처리해 매장별 원자성 보장

정산 기능의 트랜잭션 및 DB커넥션을 하나로 유지

04. / Summary

개선 및 전환 효과와 남은 과제들

기존 방식에서 이벤트 기반 적재와 Chunk 방식으로 전환하며 생긴 효과와 사이드 이펙트

개선 및 전환 효과

초기 구현 방식과 현재의 이벤트 기반 Chunk 방식과의 비교, 300개 매장 · 매장별 1500건 기준

처리 속도
(300개 매장 기준)



약 8분

약 4초

CPU 사용량



CPU 가용량의
62%

CPU 가용량의
8.4%

DB 커넥션



정산 실행 중
최대 3개

정산 실행 중
1개

고도화

서비스 운영 및 서비스 규모 증가를 고려한 고도화 지점

서비스 규모 증가에 따른
멀티스레딩 및 파티셔닝

배치 재시작 방식의 정교화

주문 완료 후 컴플레인 발생 시의
환불 정책 반영

Product & Service.

Elasticsearch

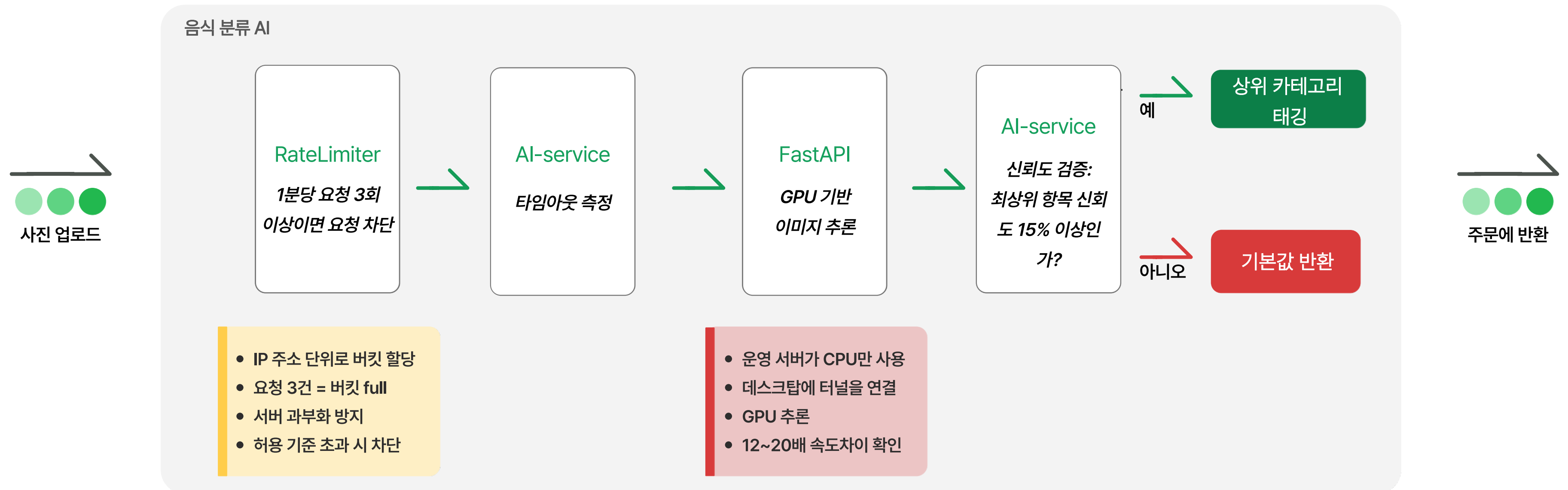
AI

판매자의 업무를 덜어줌과 동시에 수익률을 끌어올리고,
구매자에게는 검색의 신뢰도를 높이고 탐색시간을 단축시켰습니다.

01. / FastAPI-CV.

AI 기반 카테고리 분류 및 이미지 추론 파이프라인

GPU 인프라 연결 및 신뢰도 검증 분기를 통한 평균 15배 추론 속도 향상



02. / Pipeline.

Elasticsearch의 색인 & 검색 전체 파이프라인

indexing



이벤트 / Polling

매장 및 상품의 생성, 변경, 삭제에 대한 변경 감지
core의 변경사항을 스케줄러를 통해 조회

해시 비교 / 임베딩

기존 문서의 해시값을 비교하여 바뀐 필드를 선별
가게명, 메뉴명, 설명 3개 필드를 각각 벡터화

ES 저장 / 삭제

StoreDocument로 매핑하여 저장
조회 후 없거나 삭제 이벤트면 tombstone 삭제

search



쿼리 유형 판별 - BM25 실행

단순 키워드 판별
필터 표현 제외 2단어 이하

BM25 Fast-Pass

정규식으로 멀티매치 검색 수행
품질 미달 여부 검사 미달 시 LLM 파싱

LLM 파싱 / kNN 하이브리드 검색

검색어 임베딩
BM25 + kNN + 하이브리드 검색

RAG



배지 부여

요소별 가중치에 따라 정렬
조건을 넘으면 배지 부여

RAG 추천 이유 생성

왜 상위에 노출이 되었는지 이유를 알려줌
상위 5개 대상으로만 실시

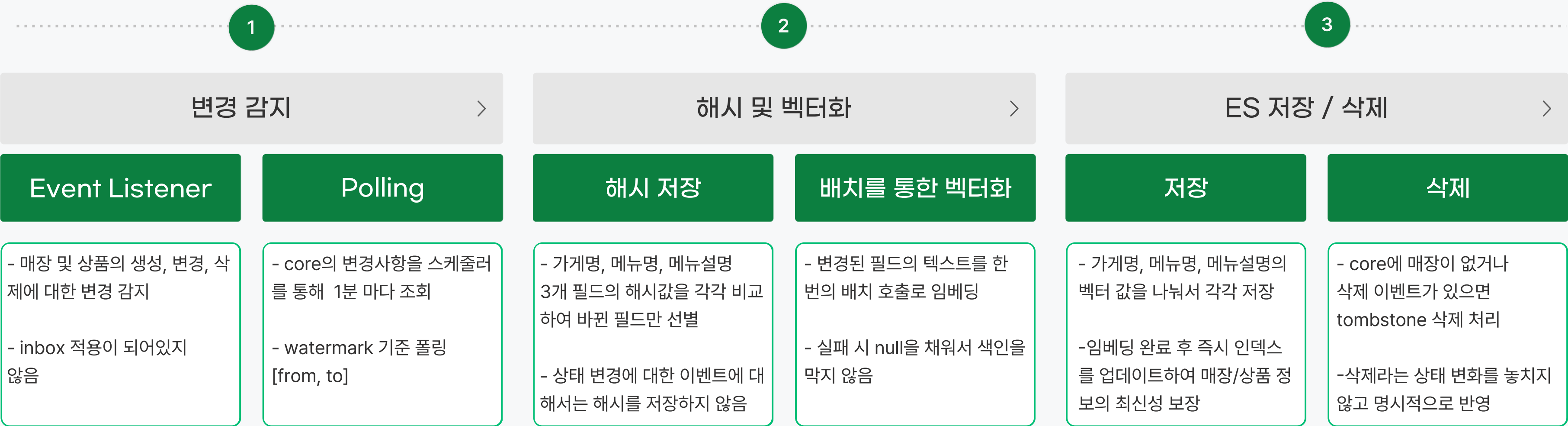
실패

기본 문장으로 대체하여 검색 자체 지연을 방지
의미 없는 검색에 대해 예외 처리

03. / Indexing.

데이터 동기화 지연을 최소화하고 검색 정확도를 높이기 위한 색인 자동화 구조

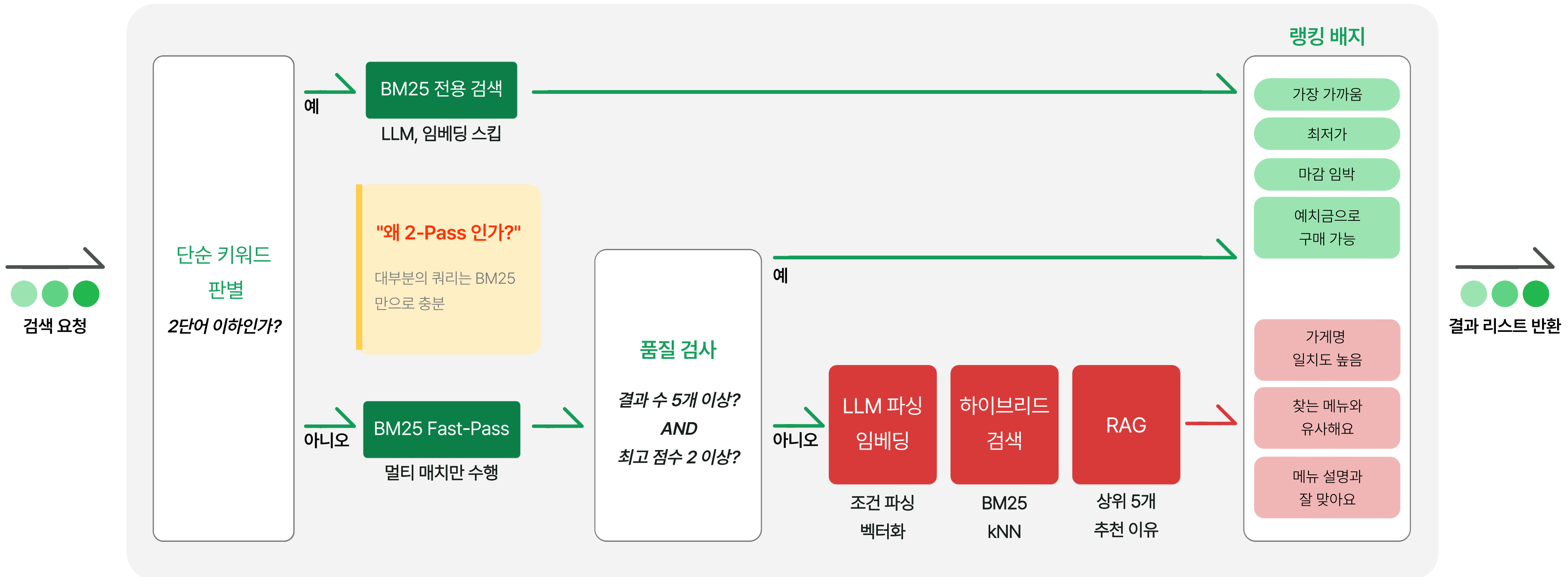
매장 및 상품 데이터 변경 사항을 실시간으로 추적하여 검색 엔진에 반영하기 위한 프로세스



04. / Searching.

검색창은 1개로

2-Pass 풀백 구조로 일반 키워드 검색과 AI 기반 검색을 한 곳에서



05. / troubleshooting.

검색 Never Die

LLM과 임베딩이 실패해도 각 계층이 예외를 흡수하여 위로 전파하지 않음



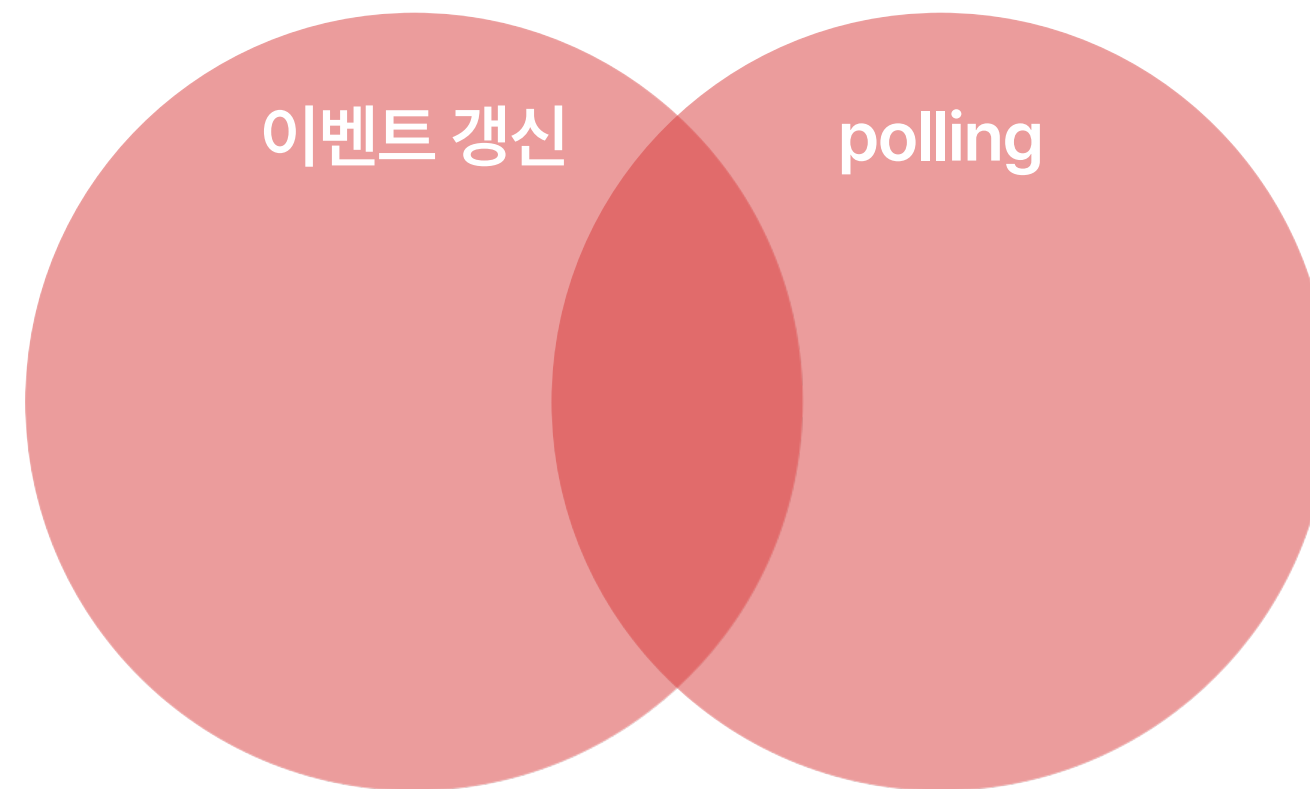
색인 시 실패한 임베딩 복구



06. / troubleshooting.

중복 재처리 방지를 통한 **비용** 최적화

이벤트로 이미 갱신된 가게가 폴링 범위에 겹치면 임베딩 API가 불필요하게 이중 호출됨



벡터와 해시의 분리

가게명, 메뉴명, 메뉴설명은 각각 1개의 벡터와 1개의
해시값을 가지고,
배치 1회 호출로 묶어 처리

텍스트 변경 체크 후 임베딩

기존 문서의 임베딩 대상 텍스트와 새로 받아온
텍스트를 해시로 비교,
해시값이 같을 경우, 기존 벡터 재사용

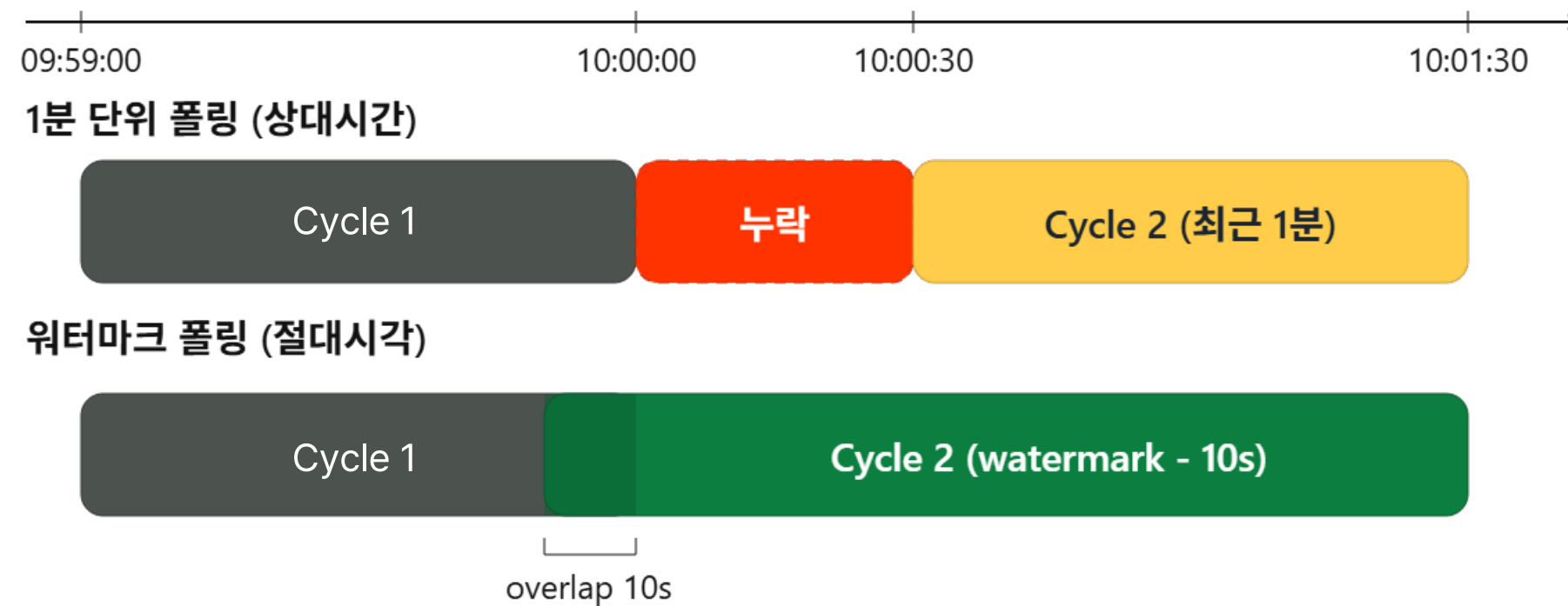
이벤트 타입별 분기

영업 상태(ON_SALE) 처럼 텍스트에 영향이 없는
이벤트는 해시 비교조차 없이 기존 벡터 재사용,
계산 자체를 생략

07. / troubleshooting.

워터마크 폴링과 Upsert 구조를 통한 데이터 **정합성** 보장

상대시간 기반 폴링이 가진 누락 위험을 절대 시각 watermark로 방어



실패 시나리오

1. 스케줄러 실행이 밀리면 최근 1분을 벗어나는 구간이 누락
2. 갱신 시각과 API 응답 사이의 시차 = Boundary condition
3. 호출 자체가 실패하면 해당 구간 통채로 유실
4. 이벤트 발행 실패
5. 동시에 도착한 이벤트 처리 중 구버전이 승리하는 경우 = Race condition

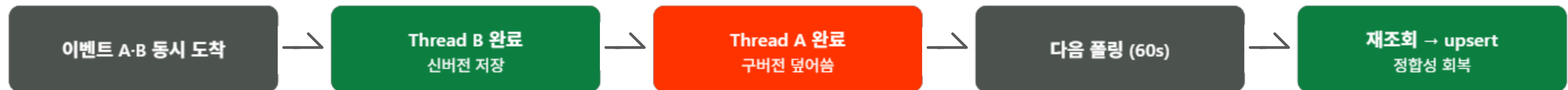
해결 시나리오

1. [from ~ to]의 watermark 폴링을 실시
2. 10초의 여유를 두고 겹치게 조회 - overlap
3. 실패 구간은 다음 polling에서 통째로 재조회
4. polling이 이벤트를 보지 않고, 원본DB의 updated_at을 직접 조회
5. 다음 polling에서 다시 upsert하며 결과적으로 정합성 회복

07. / troubleshooting.

워터마크 폴링과 Upsert 구조를 통한 데이터 **정합성** 보장

상대시간 기반 폴링이 가진 누락 위험을 절대 시각 watermark로 방어



실패 시나리오

1. 스케줄러 실행이 밀리면 최근 1분을 벗어나는 구간이 누락
2. 갱신 시각과 API 응답 사이의 시차 = Boundary condition
3. 호출 자체가 실패하면 해당 구간 통채로 유실
4. 이벤트 발행 실패
5. 동시에 도착한 이벤트 처리 중 구버전이 승리하는 경우 = Race condition

해결 시나리오

1. [from ~ to]의 watermark 폴링을 실시
2. 10초의 여유를 두고 겹치게 조회 - overlap
3. 실패 구간은 다음 polling에서 통째로 재조회
4. polling이 이벤트를 보지 않고, 원본DB의 updated_at을 직접 조회
5. 다음 polling에서 다시 upsert하며 결과적으로 정합성 회복

The Team.

CongCong_PodPod

각자 도메인을 나눠 맡고, 경계가 닿는 부분은 코드 리뷰로 서로의 영역을 이해했습니다.

깃허브 프로젝트에서 우선순위와 스프린트를 관리했고, 모든 작업은 이슈에서 시작해 PR 리뷰를 거쳐 배포까지 이어졌습니다.

PO

이동욱

기능 범위 정의와 일정 관리, 로그 체계 구축과 부하 테스트

프로젝트 기간 안에 도는 제품을 우선해 범위를 잘랐습니다. 로그를 구조화해 부하 테스트로 병목을 찾고, 같은 테스트도 조건에 따라 결과가 달라져 결론을 접은 적도 있습니다. 다시 재현되는지 확인하는 것까지가 측정이라는 걸 알았습니다.

Infra, DevOps, Event

김영진

인프라·이벤트 시스템 설계 및 개발 환경 표준화

전체 인프라와 배포 구조, Kafka 기반 공통 이벤트 모듈을 구축하고 개발환경을 표준화했습니다. 이를 통해 팀원이 도메인 개발에 집중할 수 있는 기반이 팀 전체의 생산성을 높인다는 것을 느낄 수 있었습니다.

Order, Dish, Cart

김나영

동시성 제어, 트랜잭션, 이벤트 기반 주문 비즈니스 로직 개발

주문 도메인의 비즈니스 검증과 이벤트 기반 MSA 설계, 그리고 활발한 코드·설계 리뷰를 통해 아키텍처 이해도와 제 코드의 완성도를 높이며 함께 성장하는 협업의 가치를 배울 수 있었습니다.

Payment, Deposit, Point, Level

유은지

외부 PG사 연동 및 예치금·포인트 결제 구현

데이터 정합성이 중요한 도메인들을 담당하며 비즈니스 정책부터 동시성 제어와 같은 기술적 문제까지 깊이 다뤄볼 수 있었습니다. 팀원들의 상세한 코드 리뷰 덕분에 리팩토링과 고도화를 진행할 수 있어 유익했습니다.

Store, Settlement

김민정

정산 배치 성능 개선 및 재처리 설계 및 구현

대용량 데이터 처리와 멍등 처리, 그리고 성능 테스트 및 개선을 통해 운영 중 생길 수 있는 문제들에 더 깊게 고민해볼 수 있었습니다. 또한 팀원들의 적극적인 의견 덕분에 서비스를 보다 넓은 시야로 바라볼 수 있는 좋은 기회였습니다.

Auth, Elasticsearch, AI

정지원

LLM 임베딩을 통한 ES 기반 검색과 색인 구현

BM25+kNN를 이용한 하이브리드 검색과 색인 파이프라인을 구현하면서, LLM/임베딩 장애에도 검색이 죽지 않는 구조를 구현하였고, 워터마크 폴링을 통해 정합성을 확보하는 과정에서 팀원과 함께 장애 시나리오를 검증할 수 있었습니다.

마감 재고의 가치 있는 순환

Last Dish

CongCong_PodPod

Contact. lastdish.kr

Team. Dongwook Lee(PO) / Yeongjin Kim / Nayoung Kim /
Eunji Yu / Minjung Kim / Ilwon Jung

SAVE MORE



LESS WASTE